Contents

1 Setup

I personally recommend following Section 1.2 for most seamless setup. You can always fall back to the manual setup in Section 1.3 below.

1.1 Prerequisites

You only need a **Typst** compiler, with different options for installation:

- Typst CLI
- Typst Web App (for easy collaboration with others)
- Tinymist LSP (personal recommendation)
  - one-click install extension for VSCode based editors
  - manual integration with any other LSP supporting editors

1.2 Quick start

To start with a fresh, pre-configured project, either run the following command in your terminal, assuming you have the CLI installed:

```Shell
typst init @preview/easy-hgb-thesis
```

Or sign-in to the **Typst Web App**, navigate to the easy-hgb-thesis Template on Typst Universe, and click **"Create Project in app"**.

1.3 Manual installation

First import the template package:

```Typst
#import "@preview/easy-hgb-thesis:0.2.0": full-thesis, titlepage, WORK_TYPES
```

Then set the document metadata:

```Typst
#set document(
  title: "Your Thesis Title",
  description: "A short description of your thesis",
  author: "Your Name",
  keywords: ("Keyword 1", "Keyword 2"),
```

```
)
```

Finally, wrap the document in the full-thesis template:

```typst
#show: full-thesis.with(
  titlepage: titlepage(
    "Computer Science",  // Course of study
    "Dr. Max Mentorman", // Mentor name
    work-type: WORK_TYPES.bachelor-thesis,
  ),
  kurzfassung: include "chapters/kurzfassung.typ",
  abstract: include "chapters/abstract.typ",
  bibl: bibliography("bib.yaml"), // Can be replaced with a BibLaTex file,
)

= Chapter 1

This document will explain...
```

The document is now ready to be compiled and can be customized according to your needs. Refer to Section 2 for more information.

1.4 Compilation

I recommend using the `typst compile` command of the offical **Typst CLI** binary to compile the document or tools listed in Section 3.3. This will produce a PDF file.

```shell
typst compile main.typ Your_Thesis.pdf
```

If you want to conform to specific PDF standards, you can specify the standard with the `--pdf-standard` flag.

```shell
typst compile main.typ Your_Thesis.pdf --pdf-standard a-3u --format pdf
```

For a list of available standards and their specifications, refer to the official Typst documentation.

Alternatively, you can download a compiled PDF from the **Typst Web App**.

2 Customization

The template is well documented with inline comments and docstrings. Quick exploration should reveal most customization options. An important concept are style hooks, discussed further down below in Section 2.2.

2.1 Common Options

2.1.1 Language support

The template supports **English (`en`)** and **German (`de`)**:

```typst
#set text(lang: "de")
#show: full-thesis.with(
…
)
```

2.1.2 Document metadata

Title, **description**, **authors**, and **keywords** can be customized via a `set`-rule on `document`.

```typst
#set document(
  title: "Computer Science",
  description: "A thesis on the topic of computer science",
  keywords: ("computer science", "thesis"),
  author: ("Main Author", "Co Author"),
)
#show: full-thesis.with(
…
)
```

This information is embedded in the metadata of the compiled PDF. It is also picked up by the provided `titlepage` function.

2.1.3 Titlepage

The titlepage is customizable via the `titlepage` template parameter. The template simply prepends the titlepage to the document while applying `global-style` (without `document-style`).

```typst
#show: full-thesis.with(
  titlepage: titlepage(
```

```
    "Computer Science",  // Course of study
    "Dr. Max Mentorman", // Mentor name
    work-type: WORK_TYPES.bachelor-thesis,
  ),
)
```

Omitting the titlepage is as simple as passing **`none`**:

```typst
#show: full-thesis.with(
  titlepage: none,
⋮                          …
)
```

2.1.4 Citation style

Depending on your needs, your mentor and field of work, you might need to change the citation style. Luckily, *Typst* supports pretty much all citation styles out of the box. When providing the `bibliography` template parameter, also add your desired citation style: [1]

```typst
#show: full-thesis.with(
  bibliography: bibliography("bib.yaml", style: "ieee"),
⋮                          …
)
```

[1] Harvey Lodish *et al.*, *Molecular Cell Biology*, 8th ed. 2016. Accessed: Jun. 13, 2026. [Online]. Available: https://books.google.at/books/about/Molecular_Cell_Biology.html?id=Ov7LvQEACAAJ

And switching to **APA**[1]: (Harvey Lodish et al., 2016)

```typst
#show: full-thesis.with(
  bibliography: bibliography("bib.yaml", style: "apa"),
⋮                          …
)
```

Harvey Lodish, Arnold Berk, Chris A. Kaiser, Angelika Amon, Hidde Ploegh, Anthony Bretscher, Monty Krieger, & Kelsey C. Martin. (2016). *Molecular Cell Biology* (8th ed.). https://books.google.at/books/about/Molecular_Cell_Biology.html?id=Ov7LvQEACAAJ

[1] Default citation style

2.1.5 Changing document font

Many users might want to change the font from the default base style. For this, you can use the `document-style` style hook:

```typst
#show: full-thesis.with(
  document-style: it => {
    set text(font: "Times New Roman", size: 12pt)
    it
  },
  …
)
```

2.1.5.1 Which will apply this font

To everything inside the document, even tables…

Table 1: Font in table demonstration

Header 1	Header 2
Cool	Cool Too

2.1.6 Abbreviation table

An abbreviation table is simply a Typst `dictionary` where the keys are the abbreviations and the values are the full definitions.

```typst
#let abbreviations = (
  "USB": "Universal Serial Bus",
  "RAM": "Random Access Memory",
  "MSE": "Mean Squared Error",
)
```

Then pass it to the `abbreviations` template parameter:

```typst
#show: full-thesis.with(
  abbreviations: abbreviations,
  …
)
```

Abbreviation	Description
USB	Universal Serial Bus

Abbreviation	Description
RAM	Random Access Memory
MSE	Mean Squared Error

2.1.7 Change thesis style

The template supports two different thesis styles, which control the overall look and feel of the document: **classic** and **modern**. The classic style is the default and is more traditional. It tries to loyal to the [Digital-Media/HagenbergThesis]() template, whereas the modern style is clean, uniform and inspired by higher technical college diploma thesis design guides. To change the thesis style, you can use the `thesis-style` template parameter:

```typst
#show: full-thesis.with(
  thesis-style: THESIS_STYLE.modern,
…
)
```

You can try out the different styles even after finishing your thesis (assuming no major style modifications were made by you)! Feedback to the **modern** style is especially helpful!

2.1.8 Two-column layout

Typst easily supports two-column layouts out of the box. Make use of the `content-style` style hook to achieve this:

```typst
#show: full-thesis.with(
…
  content-style: it => {
    set page(columns: 2)
    it
  },
)
```

2.1.9 Print size control box

To include a print size control box at the end of the document, which is useful when printing the document on paper but is deactivated by default for digital export, set the `include-print-size-control` template parameter to `true`:

```typst
#show: full-thesis.with(
  include-print-size-control: true,
…
```

```
)
```

2.2 Style hooks

Import `full-thesis`, then customize appearance and structure by passing style hooks to `full-thesis.with(...)`, such as:

- `global-style` (fonts, page layout)
- `document-style` (overall document)
- `content-style` (main chapters)
- Section hooks: `abstract-style`, `outline-style`, `bibliography-style`, etc.

These parameters expect functions taking the section content and returning newly styled content. For example changing the page numbering of the main content to roman numerals:

```typst
#show: full-thesis.with(
  …
  content-style: it => {
    set page(numbering: "I")
    it
  },
)
```

Defaults match typical FH thesis requirements. Override only what you need.

2.3 Sections

Most sections are optional and many can be opt-out of. Simply omitting them from the `full-thesis.with(...)` call will omit them from the document. Sections requiring explicit opt-out are specified with a `none` value:

```typst
#show: full-thesis.with(
  abstract: none,
  …
)
```

2.4 Advanced

For power users seeking fine-grained control over the document structure, the template provides individual sections as ready to use elements.

These take the section's main content as input and display a correctly stylized version of it. This allows you to put the preamble after the table of contents:

```typst
#show: full-thesis.with(
  preamble-style: _ => {
    // Replace preamble with outline
    chapter-outline()
  },
  outline-style: _ => {
    // Vice-versa
    preample-section[
      This is my preamble, which is now before the table of contents.
    ]
  }
)
```

However, by default the preamble's headings are not listed in the outline. To fix this, you can use the `style-preface` parameter:

```typst
#show: full-thesis.with(
  preamble-style: _ => {
    // Replace preamble with outline
    chapter-outline()
  },
  outline-style: _ => {
    // Vice-versa
    preample-section(style-preface: it => {
      set heading(outlined: true)
      it
    })[
      This is my preamble, which is now before the table of contents.
    ]
  }
)
```

3 Third Party Tools

3.1 Hayagriva

Hayagriva is Typst's custom bibliography tool and specification. It uses the `.yaml` format, creating a unified data model supporting over **2600** citation styles. It is the recommended way to handle citations in Typst and easy to learn, so I recommend giving it a try if you're fresh to scientific writing. You can find the documentation at https://github.com/typst/hayagriva/blob/main/docs/file-format.md.

Alternatively, Typst also supports BibLaTex files out of the box, so you can use your existing collection with the `bibliography` element.

3.2 Typstyle

Typstyle is a tool for formatting Typst code. It is my personal recommendation so you don't have to worry about formatting your source code. The documentation is available at https://typstyle-rs.github.io/typstyle/.

3.3 mise-en-place

mise is a developer environment manager, covering tools, environment variables, tasks, and more. It unifies many setups and makes new projects easy to manage. Check it out here: https://mise.jdx.dev/

While being 100% optional, for people wanting to work with mise, the quick-start approach in Section 1.2 contains a mise configuration file, which gets you going quickly. Simply executing `mise run export` will compile the document and produce a PDF file, with no manual installation of the Typst CLI or other tools required.

3.4 Packages

You will find a list of my personal recommended and useful third party Typst packages below. If you need anything not listed here, feel free to browse through Typst Universe, which provides a solution for almost any problem.

1. zero - configurable scientific number formatting; basically a requirement for scientific writing
2. cetz - semi-low-level drawing library, very powerful; often foundation for other drawing packages
3. fletcher - diagrams with nodes and arrows, powerful; easier but not as flexible as cetz
4. lilaq - easy to use plotting library with sane defaults and lots of customization options

5. codly - code-block styling and formatting, has some quite powerful features, used in this manual
6. suiji - pseudo-random number generator, useful for programmatic generation of data, solves Typst's guarantee of reproducibility and inherent statelessness

3.5 Support

If you require any assistance, are unsure about approaching certain problems, or just want to chat about Typst, feel free to reach out on GitHub, create an issue in the repository, or send me an email at timerertim@gmail.com.